

DDA LineDrawing Algorithm Implementation

```
#include <graphics.h>
#include <stdio.h>
#include <stdlib.h>

void BSALineALgo(int x1, int y1, int x2, int y2) {
    int dx = x2 - x1, dy = y2 - y1;
    int steps = abs(dx) > abs(dy) ? abs(dx) : abs(dy);
    float xIncrement = dx / (float)steps;
    float yIncrement = dy / (float)steps;
    float x = x1, y = y1;
    for (int i = 0; i <= steps; i++) {
        putpixel((int)x, (int)y, WHITE);
        x += xIncrement;
        y += yIncrement;
    }
}

void BsaWork() {
    int x1, y1, x2, y2;
    printf(" DDA LineDrawing Algorithm\n");
    printf("enter two pixel positions = x1,y1 , x2,y2 \n");
    scanf("%d %d %d %d", &x1, &y1, &x2, &y2);
    BSALineALgo(x1, y1, x2, y2);
}

int WantToRedo() {
    char c;
    printf("\n do you want to redo = y/n this is case sensitive \n ");
    scanf(" %c", &c);
    return (c == 'y' || c == 'Y');
}

int main() {
    int gd = DETECT, gm;
    initgraph(&gd, &gm, "");
    do {
        cleardevice();
        BsaWork();
    }
```

```

    } while (WantToRedo());
    closegraph();
    return 0;
}

```

OUTPUTS

```

DDA LineDrawing Algorithmn
enter two pixel positions = x1,y1 , x2,y2
10
10
10
80

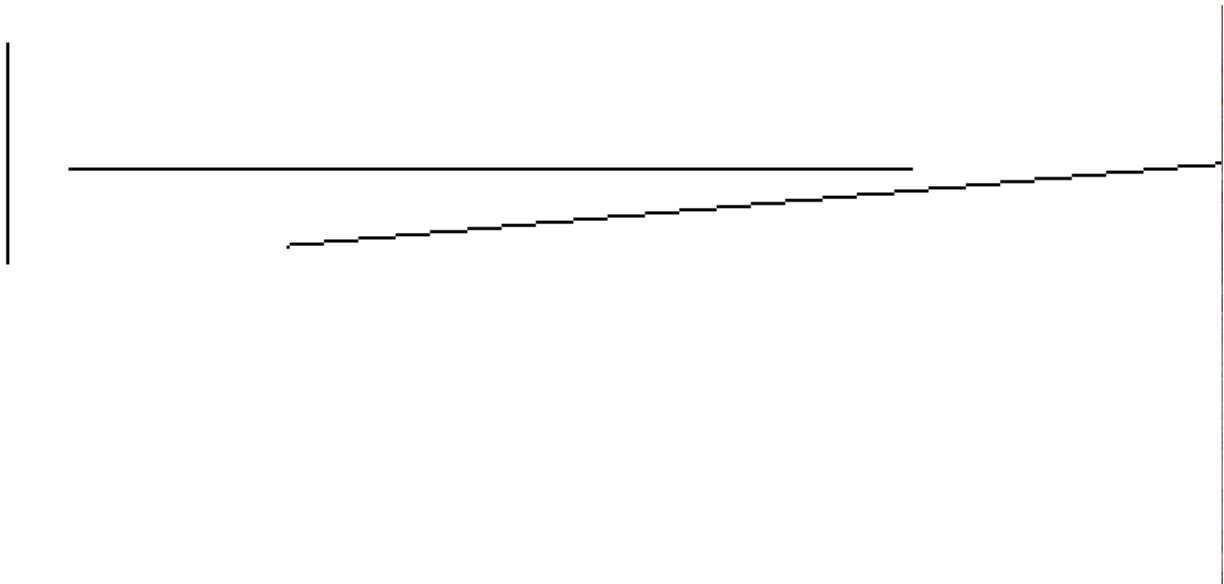
do you want to redo = y/n this is case sensitive
y
DDA LineDrawing Algorithmn
enter two pixel positions = x1,y1 , x2,y2
30
50
300
50

do you want to redo = y/n this is case sensitive
y
DDA LineDrawing Algorithmn
enter two pixel positions = x1,y1 , x2,y2
100
75
500
40

do you want to redo = y/n this is case sensitive

```

Figure



Bresenham LineDrawing Algorithm Implementation

```
#include <graphics.h>
#include <stdio.h>
#include <stdlib.h>
void BSALineALgo(int x1, int y1, int x2, int y2) {
    int dx = abs(x2 - x1), dy = abs(y2 - y1);
    int sx = (x1 < x2) ? 1 : -1; // Step direction for x
    int sy = (y1 < y2) ? 1 : -1; // Step direction for y
    int err = dx - dy;
    while (1) {
        putpixel(x1, y1, WHITE);
        if (x1 == x2 && y1 == y2)
            break;
        int e2 = 2 * err;
        if (e2 > -dy) {
            err -= dy;
            x1 += sx;
        }
        if (e2 < dx) {
            err += dx;
            y1 += sy;
        }
    }
}

void BsaWork() {
    int x1, y1, x2, y2;
    printf(" Bresenham Line Drawing Algorithm\n");
    printf("Enter two pixel positions = x1,y1 , x2,y2 \n");
    scanf("%d %d %d %d", &x1, &y1, &x2, &y2);
    BSALineALgo(x1, y1, x2, y2);
}

int WantToRedo() {
    char c;
    printf("\n Do you want to redo? (y/n) (case-sensitive)\n ");
    scanf(" %c", &c);
    return (c == 'y' || c == 'Y');
}
```

```

int main() {
    int gd = DETECT, gm;
    initgraph(&gd, &gm, "");

    do {
        cleardevice();
        BsaWork();
    } while (WantToRedo());

    closegraph();
    return 0;
}

```

OUTPUTS

```

Bresenham LineDrawing Algorithmn
enter two pixel positions = x1,y1 , x2,y2
50
70
50
400

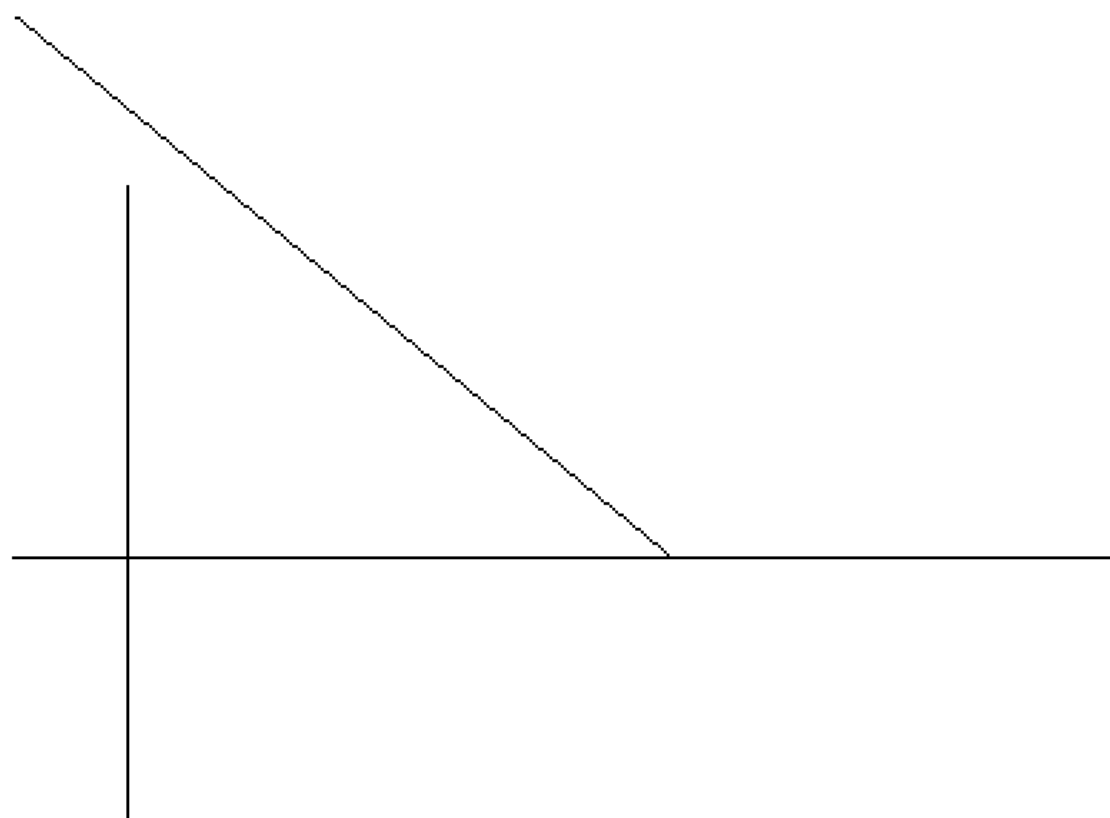
do you want to redo = y/n this is case sensitive
y
Bresenham LineDrawing Algorithmn
enter two pixel positions = x1,y1 , x2,y2
10
200
400
200

do you want to redo = y/n this is case sensitive
y
Bresenham LineDrawing Algorithmn
enter two pixel positions = x1,y1 , x2,y2
11
11
240
200

do you want to redo = y/n this is case sensitive

```

FIGURE



Circle Drawing Algorithm Implementation

```
#include <graphics.h>
#include <stdio.h>
#include <stdlib.h>

void PutPixel(int xc, int yc, int x, int y) {
    putpixel(xc + x, yc + y, WHITE);
    putpixel(xc - x, yc + y, WHITE);
    putpixel(xc + x, yc - y, WHITE);
    putpixel(xc - x, yc - y, WHITE);
    putpixel(xc + y, yc + x, WHITE);
    putpixel(xc - y, yc + x, WHITE);
    putpixel(xc + y, yc - x, WHITE);
    putpixel(xc - y, yc - x, WHITE);
}

void MidpointCircleAlgo(int xc, int yc, int r) {
    int x = 0, y = r;
    int decision = 1 - r;

    while (x <= y) {
        PutPixel(xc, yc, x, y);
        x++;
        if (decision < 0) {
            decision += 2 * x + 1;
        } else {
            y--;
            decision += 2 * (x - y) + 1;
        }
    }
}

void RunAlgo() {
    int xc, yc, r;
    printf("\nCircle Drawing Algorithm\n");
    printf("Enter center (x, y) and radius: ");
    scanf("%d %d %d", &xc, &yc, &r);
    MidpointCircleAlgo(xc, yc, r);
}
```

```
int WantToRedo() {  
    char c;  
    printf("\nDo you want to redo? (y/n) (case-sensitive): ");  
    scanf(" %c", &c);  
    return (c == 'y' || c == 'Y');  
}
```

```
int main() {  
    int gd = DETECT, gm;  
    initgraph(&gd, &gm, "");  
  
    do {  
        cleardevice();  
        RunAlgo();  
    } while (WantToRedo());  
  
    closegraph();  
    return 0;  
}
```

OUTPUTS

```
Circle Drawing Algorithmn
enter  centre = x1,y1 and radius r
100
100
60

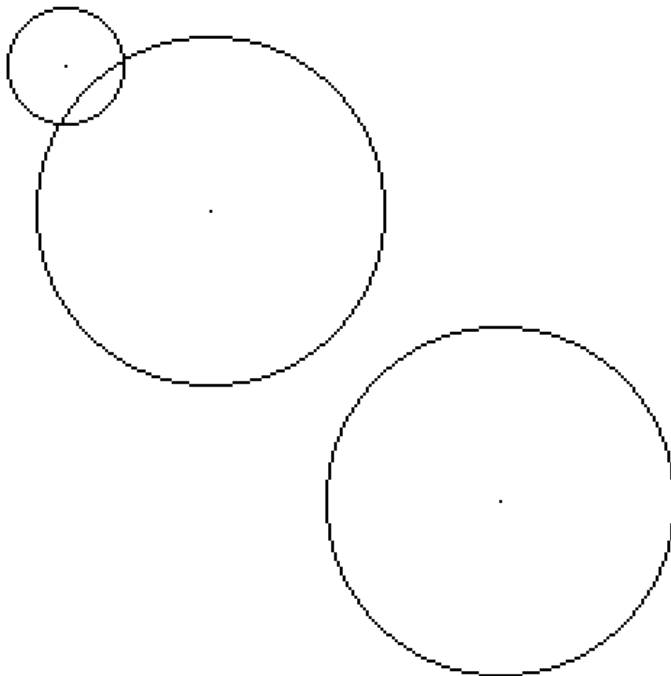
do you want to redo = y/n this is case sensitive
y

Circle Drawing Algorithmn
enter  centre = x1,y1 and radius r
200
200
60

do you want to redo = y/n this is case sensitive
y

Circle Drawing Algorithmn
enter  centre = x1,y1 and radius r
50
50
20

do you want to redo = y/n this is case sensitive
```



Flood Fill Algorithm Implementation

```
#include <conio.h>
#include <graphics.h>
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int x, y;
} Position;

typedef struct {
    int left, top, right, bottom;
} Rectangle;

typedef struct PixelData {
    Position position;
    int alreadyVisited;
    int alreadyPushed;
} PixelData;

Rectangle rectanglePosition;
Rectangle rectangleSize;
Position initialPos;
PixelData pixels[640 * 480];
int pixelCount = 0;
Position stack[640 * 480];
int stackTop = -1;

int GetPixelIndex(int x, int y) {
    return y * 640 + x;
}

PixelData* GetPixel(int index) {
    return &pixels[index];
}
```

```

void FloodFillStart() {
    printf("\nFlood Fill Algorithm\n");
    printf("Enter rectangle position and size = x, y, w, h\n");
    scanf("%d %d %d %d", &rectanglePosition.left, &rectanglePosition.top, &rectangleSize.right,
    &rectangleSize.bottom);

    initialPos.x = (rectanglePosition.left + rectangleSize.right) / 2;
    initialPos.y = (rectanglePosition.top + rectangleSize.bottom) / 2;
    stack[++stackTop] = initialPos;
    FloodFillAlgoStart();
}

void FloodFillAlgoStart() {
    while (stackTop >= 0) {
        Position top = stack[stackTop--];
        PixelData* pixel = GetPixel(GetPixelIndex(top.x, top.y));
        if (!pixel->alreadyVisited) {
            FillThePixel(pixel);
            Propagate(pixel);
        }
    }
}

int CheckIsValid(int x, int y) {
    return (x >= rectanglePosition.left && x < rectanglePosition.left + rectangleSize.right) &&
        (y >= rectanglePosition.top && y < rectanglePosition.top + rectangleSize.bottom);
}

void FillThePixel(PixelData* pixel) {
    pixel->alreadyVisited = 1;
    setcolor(BLACK);
    putpixel(pixel->position.x, pixel->position.y, BLACK);
}

```

```

void Propagate(PixelData* pixel) {
    Position position = pixel->position;
    for (int i = -1; i <= 1; ++i) {
        for (int j = -1; j <= 1; ++j) {
            Position newPos = { position.x + i, position.y + j };
            if (CheckIsValid(newPos.x, newPos.y)) {
                PixelData* newPixel = GetPixel(GetPixelIndex(newPos.x, newPos.y));
                if (!newPixel->alreadyVisited && !newPixel->alreadyPushed) {
                    newPixel->alreadyPushed = 1;
                    stack[++stackTop] = newPos;
                }
            }
        }
    }
}

```

```

int main() {
    int gd = DETECT, gm;
    initgraph(&gd, &gm, "");

    FloodFillStart();

    char c;
    printf("\nDo you want to redo = y/n (case sensitive)\n");
    scanf(" %c", &c);
    if (c == 'y' || c == 'Y') {
        FloodFillStart();
    }

    closegraph();
    return 0;
}

```

OUTPUTS

```
Flood Fill Algorithmn
enter  rectangle position and size = x,y , w ,h
100
100
100
100

do you want  to redo = y/n  this is case sensitive
y
```

```
Flood Fill Algorithmn
enter  rectangle position and size = x,y , w ,h
50
50
70
50

do you want  to redo = y/n  this is case sensitive
y
```

```
Flood Fill Algorithmn
enter  rectangle position and size = x,y , w ,h
250
100
30
200
```



Quadrilateral Drawing Algorithm Implementation

```
#include <conio.h>
#include <graphics.h>
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int x, y;
} Position;

Position position1, position2, position3, position4;

void DrawQuadrilateral() {
    printf("Quadrilateral Drawing Algorithm\n");

    printf("\nEnter pixel positions 1 = (x1, y1)\n");
    scanf("%d %d", &position1.x, &position1.y);
    printf("\nEnter pixel positions 2 = (x2, y2)\n");
    scanf("%d %d", &position2.x, &position2.y);
    printf("\nEnter pixel positions 3 = (x3, y3)\n");
    scanf("%d %d", &position3.x, &position3.y);
    printf("\nEnter pixel positions 4 = (x4, y4)\n");
    scanf("%d %d", &position4.x, &position4.y);
    QuadrilateralAlgo(position1, position2, position3, position4);
}

void QuadrilateralAlgo(Position p1, Position p2, Position p3, Position p4) {
    LineDraw(p1, p2);
    LineDraw(p2, p3);
    LineDraw(p3, p4);
    LineDraw(p4, p1);
}

void LineDraw(Position p1, Position p2) {
    line(p1.x, p1.y, p2.x, p2.y);
}
```

```

int main() {
    int gd = DETECT, gm;
    initgraph(&gd, &gm, "");
    DrawQuadrilateral();
    char c;
    printf("\nDo you want to redo = y/n (case sensitive)\n");
    scanf(" %c", &c);
    if (c == 'y' || c == 'Y') {
        DrawQuadrilateral();
    }
    closegraph();
    return 0;
}

```

OUTPUTS

Quadrilateral Drawing Algorithm

```

enter pixel positions1 = (x1,y1)
10
60

```

```

enter pixel positions2 = (x2,y2)
50
10

```

```

enter pixel positions3 = ((x3,y3)
100
60

```

```

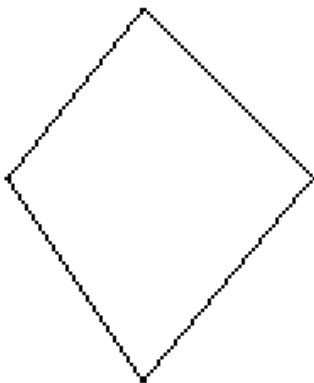
enter pixel positions4 = (x4,y4))
50
120

```

```

do you want to redo = y/n this is case sensitive

```



Quadrilateral Translation Implementation

```
#include <conio.h>
#include <graphics.h>
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int x, y;
} Position;
Position position1, position2, position3, position4, TranslationVector;

void TranslateQuad() {
    printf("\nEnter pixel positions 1 = (x1, y1)\n");
    scanf("%d %d", &position1.x, &position1.y);
    printf("\nEnter pixel positions 2 = (x2, y2)\n");
    scanf("%d %d", &position2.x, &position2.y);
    printf("\nEnter pixel positions 3 = (x3, y3)\n");
    scanf("%d %d", &position3.x, &position3.y);
    printf("\nEnter pixel positions 4 = (x4, y4)\n");
    scanf("%d %d", &position4.x, &position4.y);
    printf("\nEnter translating vector = (x, y)\n");
    scanf("%d %d", &TranslationVector.x, &TranslationVector.y);
    AlgoTranslateQuad(position1, position2, position3, position4, TranslationVector);
}

void AlgoTranslateQuad(Position p1, Position p2, Position p3, Position p4, Position tVector) {
    QuadrilateralAlgo(p1, p2, p3, p4);
    QuadrilateralAlgo((Position){p1.x + tVector.x, p1.y + tVector.y},
        (Position){p2.x + tVector.x, p2.y + tVector.y},
        (Position){p3.x + tVector.x, p3.y + tVector.y},
        (Position){p4.x + tVector.x, p4.y + tVector.y});
}

void QuadrilateralAlgo(Position p1, Position p2, Position p3, Position p4) {
    LineDraw(p1, p2);
    LineDraw(p2, p3);
    LineDraw(p3, p4);
    LineDraw(p4, p1);
}
```

```

void LineDraw(Position p1, Position p2) {
    line(p1.x, p1.y, p2.x, p2.y);
}

int main() {
    int gd = DETECT, gm;
    initgraph(&gd, &gm, "");
    TranslateQuad();
    char c;
    printf("\nDo you want to redo = y/n (case sensitive)\n");
    scanf(" %c", &c);
    if (c == 'y' || c == 'Y') {
        TranslateQuad();
    }
    closegraph();
    return 0;
}

```

OUTPUTS

```

enter pixel positions1 = (x1,y1)
10
60

enter pixel positions2 = (x2,y2)
50
10

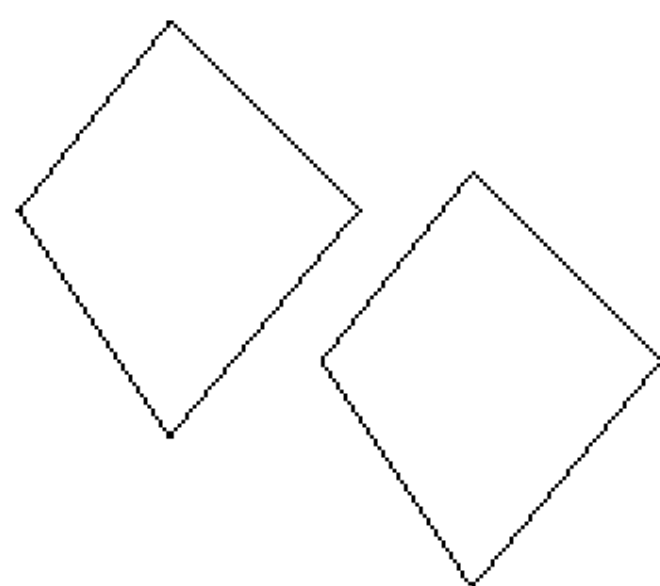
enter pixel positions3 = ((x3,y3)
100
60

enter pixel positions4 = (x4,y4))
50
120

enter translating vector = (x,y))
80
40

do you want to redo = y/n this is case sensitive

```



Rotation Algorithm Implementation

```
#include <conio.h>
#include <graphics.h>
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

#define PI 3.14159265

typedef struct {
    int x, y;
} Position;

Position position1, position2, position3, position4;
float angle;

Position Rotate(Position p, float radAngle) {
    Position rotated;
    rotated.x = p.x * cos(radAngle) - p.y * sin(radAngle);
    rotated.y = p.x * sin(radAngle) + p.y * cos(radAngle);
    return rotated;
}

void RotateQuad() {
    printf("Rotation Algorithm\n");

    printf("\nEnter pixel positions 1 = (x1, y1)\n");
    scanf("%d %d", &position1.x, &position1.y);
    printf("\nEnter pixel positions 2 = (x2, y2)\n");
    scanf("%d %d", &position2.x, &position2.y);
    printf("\nEnter pixel positions 3 = (x3, y3)\n");
    scanf("%d %d", &position3.x, &position3.y);
    printf("\nEnter pixel positions 4 = (x4, y4)\n");
    scanf("%d %d", &position4.x, &position4.y);
    printf("\nEnter rotation angle in degree\n");
    scanf("%f", &angle);

    AlgoRotateQuad(position1, position2, position3, position4, angle);
}

void AlgoRotateQuad(Position p1, Position p2, Position p3, Position p4, float angle) {
    QuadrilateralAlgo(p1, p2, p3, p4);
    DoCalculationAndRotate(p1, p2, p3, p4, angle);
}
```

```
}
```

```
void DoCalculationAndRotate(Position p1, Position p2, Position p3, Position p4, float angle) {  
    float radAngle = angle * PI / 180.0f;  
    Position mid = {(p1.x + p2.x + p3.x + p4.x) / 4, (p1.y + p2.y + p3.y + p4.y) / 4};  
  
    Position rotatedposition1 = Rotate(p1, radAngle);  
    rotatedposition1.x += mid.x;  
    rotatedposition1.y += mid.y;  
  
    Position rotatedposition2 = Rotate(p2, radAngle);  
    rotatedposition2.x += mid.x;  
    rotatedposition2.y += mid.y;  
  
    Position rotatedposition3 = Rotate(p3, radAngle);  
    rotatedposition3.x += mid.x;  
    rotatedposition3.y += mid.y;  
  
    Position rotatedposition4 = Rotate(p4, radAngle);  
    rotatedposition4.x += mid.x;  
    rotatedposition4.y += mid.y;  
  
    QuadrilateralAlgo(rotatedposition1, rotatedposition2, rotatedposition3, rotatedposition4);  
}
```

```
void QuadrilateralAlgo(Position p1, Position p2, Position p3, Position p4) {  
    LineDraw(p1, p2);  
    LineDraw(p2, p3);  
    LineDraw(p3, p4);  
    LineDraw(p4, p1);  
}
```

```
void LineDraw(Position p1, Position p2) {  
    line(p1.x, p1.y, p2.x, p2.y);  
}
```

```
int main() {  
    int gd = DETECT, gm;  
    initgraph(&gd, &gm, "");  
  
    RotateQuad();  
  
    char c;  
    printf("\nDo you want to redo = y/n (case sensitive)\n");
```

```

scanf(" %c", &c);
if (c == 'y' || c == 'Y') {
    RotateQuad();
}

closegraph();
return 0;
}

```

OUTPUTS

```

Roation Algorithmn

enter pixel positions1 = (x1,y1) = 10
60

enter pixel positions2 = (x2,y2) = 50
10

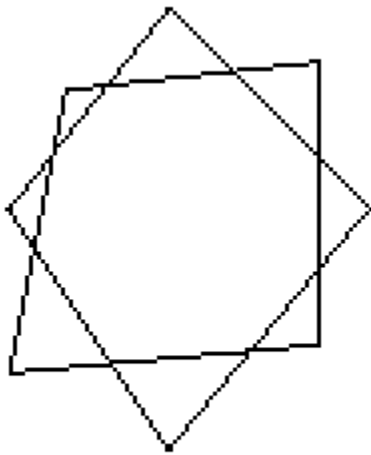
enter pixel positions3 = ((x3,y3) = 100
60

enter pixel positions4 = (x4,y4)) = 50
120

enter rotation angle in degree = 45

do you want to redo = y/n this is case sensitive

```



Reflection Around A Point Algorithm Implementation

```
#include <conio.h>
#include <graphics.h>
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

typedef struct {
    int x, y;
} Position;

Position position1, position2, position3, position4, reflectionPoint;

Position Reflect(Position p, Position reflectPoint) {
    Position reflected;
    reflected.x = reflectPoint.x - (p.x - reflectPoint.x);
    reflected.y = reflectPoint.y - (p.y - reflectPoint.y);
    return reflected;
}

void ReflectQuadAboutPoint() {
    printf("Reflection Along A Point Algorithm\n");

    printf("\nEnter pixel positions 1 = (x1, y1)\n");
    scanf("%d %d", &position1.x, &position1.y);
    printf("\nEnter pixel positions 2 = (x2, y2)\n");
    scanf("%d %d", &position2.x, &position2.y);
    printf("\nEnter pixel positions 3 = (x3, y3)\n");
    scanf("%d %d", &position3.x, &position3.y);
    printf("\nEnter pixel positions 4 = (x4, y4)\n");
    scanf("%d %d", &position4.x, &position4.y);
    printf("\nEnter point along which rectangle is going to be reflected\n");
    scanf("%d %d", &reflectionPoint.x, &reflectionPoint.y);

    AlgoReflectQuad(position1, position2, position3, position4, reflectionPoint);
}
```

```

void AlgoReflectQuad(Position p1, Position p2, Position p3, Position p4, Position reflectPoint) {
    QuadrilateralAlgo(p1, p2, p3, p4);
    singleTonPixels.PutPixel(reflectPoint.x, reflectPoint.y, BLACK);
    DoCalculationAndReflectQuadAboutPoint(p1, p2, p3, p4, reflectPoint);
}

```

```

void DoCalculationAndReflectQuadAboutPoint(Position p1, Position p2, Position p3, Position
p4, Position reflectPoint) {
    Position reflectedposition1 = Reflect(p1, reflectPoint);
    Position reflectedposition2 = Reflect(p2, reflectPoint);
    Position reflectedposition3 = Reflect(p3, reflectPoint);
    Position reflectedposition4 = Reflect(p4, reflectPoint);

    QuadrilateralAlgo(reflectedposition1, reflectedposition2, reflectedposition3,
reflectedposition4);
}

```

```

void QuadrilateralAlgo(Position p1, Position p2, Position p3, Position p4) {
    LineDraw(p1, p2);
    LineDraw(p2, p3);
    LineDraw(p3, p4);
    LineDraw(p4, p1);
}

```

```

void LineDraw(Position p1, Position p2) {
    line(p1.x, p1.y, p2.x, p2.y);
}

```

```

int main() {
    int gd = DETECT, gm;
    initgraph(&gd, &gm, "");

    ReflectQuadAboutPoint();

    char c;
    printf("\nDo you want to redo = y/n (case sensitive)\n");
    scanf(" %c", &c);
    if (c == 'y' || c == 'Y') {
        ReflectQuadAboutPoint();
    }

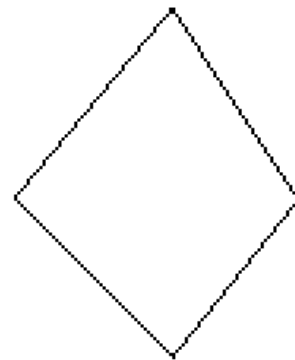
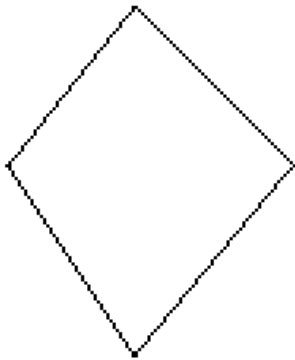
    closegraph();
    return 0;
}

```

```
}
```

OUTPUTS

```
Reflection Along A Point Algorithmn  
  
enter pixel positions1 = (x1,y1) = 10  
60  
  
enter pixel positions2 = (x2,y2) = 50  
10  
  
enter pixel positions3 = ((x3,y3) = 100  
60  
  
enter pixel positions4 = (x4,y4)) = 50  
120  
  
enter point along which rect is going to be reflected = 150  
100  
  
do you want to redo = y/n this is case sensitive
```



Reflecting Along A Line Implementation

```
#include <graphics.h>
#include <math.h>
#include <stdio.h>

typedef struct {
    int x, y;
} Position;

Position position1, position2, position3, position4;
float slope, yInter;

void ReflectQuadAboutLine() {
    printf("\nEnter pixel positions 1 = (x1, y1)\n");
    scanf("%d %d", &position1.x, &position1.y);
    printf("\nEnter pixel positions 2 = (x2, y2)\n");
    scanf("%d %d", &position2.x, &position2.y);
    printf("\nEnter pixel positions 3 = (x3, y3)\n");
    scanf("%d %d", &position3.x, &position3.y);
    printf("\nEnter pixel positions 4 = (x4, y4)\n");
    scanf("%d %d", &position4.x, &position4.y);
    printf("\nEnter line slope and y-intercept\n");
    scanf("%f %f", &slope, &yInter);

    AlgoRotateQuad(position1, position2, position3, position4, slope, yInter);
}

void AlgoRotateQuad(Position p1, Position p2, Position p3, Position p4, float slope, float yinter) {
    QuadrilateralAlgo(p1, p2, p3, p4);
    DrawTheLine(slope, yinter);
    DoCalculationAndReflectQuadAboutLine(p1, p2, p3, p4, slope, yinter);
}

void QuadrilateralAlgo(Position p1, Position p2, Position p3, Position p4) {
    LineDraw(p1, p2);
    LineDraw(p2, p3);
    LineDraw(p3, p4);
    LineDraw(p4, p1);
}
```

```
}
```

```
void LineDraw(Position p1, Position p2) {  
    line(p1.x, p1.y, p2.x, p2.y);  
}
```

```
void DoCalculationAndReflectQuadAboutLine(Position p1, Position p2, Position p3, Position p4,  
float slope, float yinter) {  
    float OneMinusMsquare = 1 - slope * slope;  
    float OnePlusMsquare = 1 + slope * slope;  
    float twoM = slope + slope;  
    float MinusTwoC_M = -2 * yinter * slope;  
    float twoC = yinter + yinter;
```

```
    Position reflectedPosition1 = Reflect(p1, OneMinusMsquare, OnePlusMsquare, twoM,  
MinusTwoC_M, twoC);
```

```
    Position reflectedPosition2 = Reflect(p2, OneMinusMsquare, OnePlusMsquare, twoM,  
MinusTwoC_M, twoC);
```

```
    Position reflectedPosition3 = Reflect(p3, OneMinusMsquare, OnePlusMsquare, twoM,  
MinusTwoC_M, twoC);
```

```
    Position reflectedPosition4 = Reflect(p4, OneMinusMsquare, OnePlusMsquare, twoM,  
MinusTwoC_M, twoC);
```

```
    QuadrilateralAlgo(reflectedPosition1, reflectedPosition2, reflectedPosition3,  
reflectedPosition4);  
}
```

```
void DrawTheLine(float slope, float ycoeff) {  
    Position linePoint1 = {0, ycoeff};  
    Position linePoint2 = {getmaxx(), slope * getmaxx() + ycoeff};  
    LineDraw(linePoint1, linePoint2);  
}
```

```
Position Reflect(Position point, float OneMinusMsquare, float OnePlusMsquare, float twoM, float  
MinusTwoC_M, float twoC) {
```

```
    float A = (OneMinusMsquare * point.x + twoM * point.y + MinusTwoC_M) /  
(OnePlusMsquare);
```

```
    float B = (twoM * point.x - OneMinusMsquare * point.y + twoC) / (OnePlusMsquare);
```

```
    Position reflectedPoint = {A, B};
```

```
    return reflectedPoint;
```

```
}
```

```
int main() {  
    int gd = DETECT, gm;
```

```

initgraph(&gd, &gm, "");

ReflectQuadAboutLine();

char c;
printf("\nDo you want to redo = y/n (case sensitive)\n");
scanf(" %c", &c);
if (c == 'y' || c == 'Y') {
    ReflectQuadAboutLine();
}

closegraph();
return 0;
}

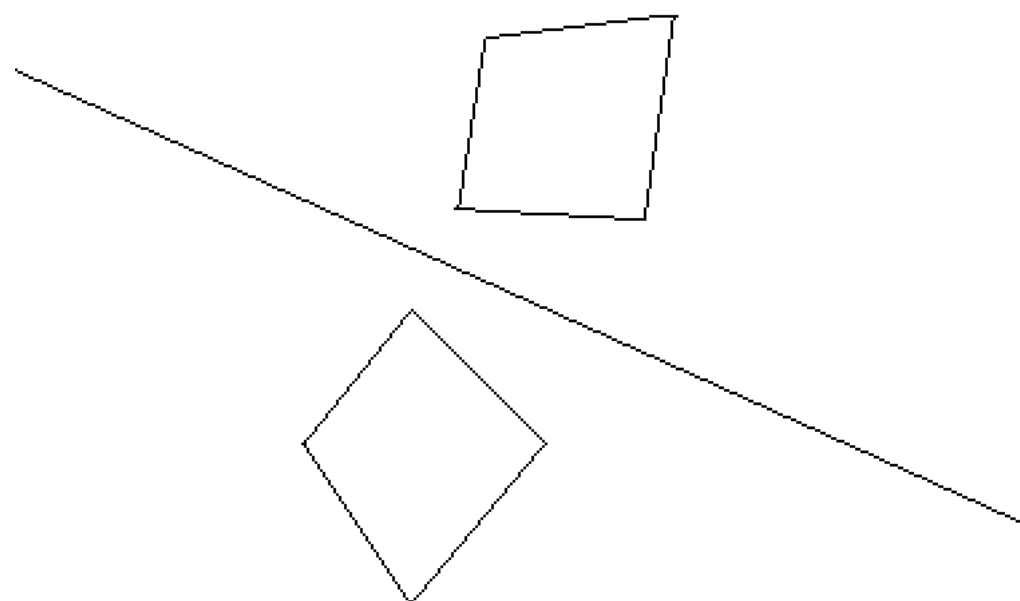
```

OUTPUTS

```

Reflection Along A Line Algorithmn
enter pixel positions1 = (x1,y1) = 110
160
enter pixel positions2 = (x2,y2) = 150
110
enter pixel positions3 = ((x3,y3) = 200
160
enter pixel positions4 = (x4,y4)) = 150
220
enter line slope and yintercept = .45
20
do you want to redo = y/n this is case sensitive

```



Quadrilateral Scaling Implementation

```
#include <graphics.h>
#include <stdio.h>

typedef struct {
    int x, y;
} Position;

Position position1, position2, position3, position4;
Position scalingFactors;

void ScaleQuad() {
    printf("\nEnter pixel positions 1 = (x1, y1)\n");
    scanf("%d %d", &position1.x, &position1.y);
    printf("\nEnter pixel positions 2 = (x2, y2)\n");
    scanf("%d %d", &position2.x, &position2.y);
    printf("\nEnter pixel positions 3 = (x3, y3)\n");
    scanf("%d %d", &position3.x, &position3.y);
    printf("\nEnter pixel positions 4 = (x4, y4)\n");
    scanf("%d %d", &position4.x, &position4.y);
    printf("\nEnter scaling factors (Sx, Sy)\n");
    scanf("%d %d", &scalingFactors.x, &scalingFactors.y);

    AlgoScaleQuad(position1, position2, position3, position4, scalingFactors);
}

void AlgoScaleQuad(Position p1, Position p2, Position p3, Position p4, Position scalingFactor) {
    QuadrilateralAlgo(p1, p2, p3, p4);
    DoCalculationAndScale(p1, p2, p3, p4, scalingFactor);
}

void QuadrilateralAlgo(Position p1, Position p2, Position p3, Position p4) {
    LineDraw(p1, p2);
    LineDraw(p2, p3);
    LineDraw(p3, p4);
    LineDraw(p4, p1);
}

void LineDraw(Position p1, Position p2) {
```

```
    line(p1.x, p1.y, p2.x, p2.y);  
}
```

```
void DoCalculationAndScale(Position p1, Position p2, Position p3, Position p4, Position  
scalingFactor) {
```

```
    Position mid = {(p1.x + p2.x + p3.x + p4.x) / 4, (p1.y + p2.y + p3.y + p4.y) / 4};
```

```
    Position scaledPosition1 = Scale(p1, mid, scalingFactor);
```

```
    Position scaledPosition2 = Scale(p2, mid, scalingFactor);
```

```
    Position scaledPosition3 = Scale(p3, mid, scalingFactor);
```

```
    Position scaledPosition4 = Scale(p4, mid, scalingFactor);
```

```
    QuadrilateralAlgo(scaledPosition1, scaledPosition2, scaledPosition3, scaledPosition4);  
}
```

```
Position Scale(Position point, Position mid, Position scalingFactor) {
```

```
    Position scaledPoint;
```

```
    scaledPoint.x = mid.x + (point.x - mid.x) * scalingFactor.x;
```

```
    scaledPoint.y = mid.y + (point.y - mid.y) * scalingFactor.y;
```

```
    return scaledPoint;
```

```
}
```

```
int main() {
```

```
    int gd = DETECT, gm;
```

```
    initgraph(&gd, &gm, "");
```

```
    ScaleQuad();
```

```
    char c;
```

```
    printf("\nDo you want to redo = y/n (case sensitive)\n");
```

```
    scanf(" %c", &c);
```

```
    if (c == 'y' || c == 'Y') {
```

```
        ScaleQuad();
```

```
    }
```

```
    closegraph();
```

```
    return 0;
```

```
}
```

OUTPUTS

Scaling Algorithmn

enter pixel positions1 = (x1,y1) = 110
150

enter pixel positions2 = (x2,y2) = 150
110

enter pixel positions3 = ((x3,y3) = 100
60

enter pixel positions4 = (x4,y4)) = 50
120

enter Scaling factors (Sx,Sy) = 1.8
1.8

do you want to redo = y/n this is case sensitive

